Laboratoire expérimental AAM O94.6

Protocole, essais reproduisibles et interprétation des résultats

20 octobre 2026

Contents

1 Question scientifique et périmètre **1**

2 Répertoire de preuve et méthode anti-contamination **1**

3 Phase P0 — reproduire la référence O94.5 **2**

4 Phase P1 — pertinence RAG par entité **2**

5 Phase P2 — multi-question et synchronisation du chat **2**

6 Phase P3 — génération longue et gestion des délais **2**

7 Phase P4 — composition, compilation et éditeur multi-fichiers **2**

8 Phase P5 — observation d'un processus de facturation **3**

9 Phase P6 — résolution continue et récompense **3**

10 Essais complémentaires P7 — sorties brutes du chat **3**
10.1 RAG sans modèle intégré 3
10.2 Fournisseur intégré et RAG 3
10.3 Modèle de test sans RAG 3
10.4 Trois questions dans un seul prompt 3
10.5 Exemple demandé: capitale du Japon 3

11 Phase P8 — restauration et paquetage **4**

12 Commandes de reproduction **4**

13 Conclusion et état **4**

1 Question scientifique et périmètre

Ce laboratoire n'affirme pas d'avance que AAM réussira ou échouera. Il définit des épreuves réelles, garde les sources et oracles séparés, conserve les observations brutes et analyse chaque écart. L'objectif opérationnel est de tester le passage entre observation, recherche, modèle intégré, composition, compilation, résolution itérative et chat.

2 Répertoire de preuve et méthode anti-contamination

1. Les entrées d'essai sont écrites dans `real_task_tests/O94_6_replay/`; les oracles d'acceptation sont dans `real_task_tests/O94_6_replay/oracles/` et ne sont pas chargés par AAMRuntime.

2. Avant un run, enregistrer versions, commandes, temps, hashes, état des sources RAG et snapshot.
3. Capturer stdout, stderr, code de sortie, statut, sources retournées et trace mémoire.
4. Garder tout échec, timeout ou test bloqué comme événement distinct; ne jamais transformer automatiquement `UNRESOLVED` ou `TIMEOUT` en succès.
5. Faire tourner les tests UI sous Xvfb dans ce conteneur; une validation locale côté utilisateur se fait sur sa machine, qui n'est pas accessible à distance depuis cette session.

3 Phase P0 — reproduire la référence O94.5

La baseline a été réexécutée et conservée dans les reçus P0. Elle comportait des confusions d'entité: par exemple, une question sur GB-42 produisait une réponse de JP-19. Une requête multi-question répondait à seulement une des trois sous-questions. Ces résultats historiques restent intactes dans `real_task_tests/O94_5_baseline/evidence/`.

4 Phase P1 — pertinence RAG par entité

Jeu d'entités indépendantes GB-42 et FR-27. Sept prompts historiques sont évalués avec un oracle indépendant. O94.6 rend sept réponses correctes sur sept dans la campagne d'évaluation; exemples: GB-42 est affectée à Warehouse South, consigne -12 C, cycle 21 jours; FR-27 est Warehouse North, consigne -8 C, cycle 14 jours; Q7.4 demande isolation si la température dépasse -10 C pendant plus de six minutes et notification de l'ingénieur de garde. La source porte un hash et la sélection de passage est enregistrée. Ce test mesure la recherche/extraction depuis le corpus, pas la qualité d'un LLM.

5 Phase P2 — multi-question et synchronisation du chat

Une seule requête demande le dépôt, le point de consigne et la fréquence d'inspection pour GB-42. Trois enfants sont créés et donnent trois sorties ordonnées avec leurs preuves. Une deuxième question demande les attributs de VX-318 et attend Juniper, -11.5 C et 17 jours. L'oracle est uniquement utilisé après le retour d'AAM.

6 Phase P3 — génération longue et gestion des délais

Le scénario de sortie longue construit 600 segments et vérifie conservation complète et visibilité des marqueurs de début/fin. La réponse complète possède 58 199 caractères, hash `fa0c90e8de700034df9f1a935864e6c` durée observée de 3,1804 s et un transcript visible de 58 443 caractères. Le fournisseur est une fixture de protocole déterministe, pas un LLM. Des tests séparés vérifient timeout réseau explicite, absence de modèle et erreurs fournisseur visibles.

7 Phase P4 — composition, compilation et éditeur multi-fichiers

La composition JSON de contrôle qualité contient dix nœuds et trois catégories de briques validées. Elle compile en code source Python et bytecode CPython puis s'exécute sur un projet multi-fichiers; les tests cible passent. Le programme de maintenance traite quatre objets et le vérificateur indépendant recalcule les jours de retard et les classes d'action. Le mini-éditeur multi-fichiers produit par le scénario compile quatre modules sans échec et passe cinq tests. Une variante modifiée change le comportement et est recompilée. Les hashes sont dans `real_task_tests/O94_6_replay/evidence/`.

8 Phase P5 — observation d'un processus de facturation

Le programme cible exécute validation de schéma, normalisation de lignes, remise, frais d'expédition, TVA avec arrondi décimal et finalisation. La capture l'exécute en sous-processus, sans importer le code cible dans le moteur. Huit paires sont observées; six appartiennent à l'entraînement et se rejouent exactement, deux sont gardées à l'écart et restent `UNRESOLVED`. Les checkpoints et la capsule sont stockés. Le réplicateur n'a pas généralisé à ces deux entrées: cette observation est un résultat de test, pas un échec présumé du potentiel général d'AAM.

9 Phase P6 — résolution continue et récompense

Le runtime résout une tâche depuis un passage local, puis un vérificateur indépendant valide une contrainte après la sortie. Les traces établissent une provenance source et un événement de récompense. Les tests montrent des dimensions distinctes pour retour humain, réussite opérationnelle et vérité vérifiée; une erreur fournisseur pénalise la fiabilité de la capacité de génération, tandis que le statut non résolu ne devient pas une assertion factuelle.

10 Essais complémentaires P7 — sorties brutes du chat

10.1 RAG sans modèle intégré

Prompt: `Which warehouse is assigned to service card XD-620?`. Réponse exacte: `Service card XD-620: the assigned warehouse is Willow Quay.` Hash de réponse: `b34c625599e193637221436e557`

10.2 Fournisseur intégré et RAG

Prompt: `What is the set point for service card XD-620?`. Réponse brute: `Generated from supplied test context: [[P7_independent_service_card_source.txt#0:9867ac27e6]] Service card XD-620: the assigned warehouse is Willow Quay. The set point is -14.5 C.` Hash: `27a8d809e411632532d9afade90b72535367e3c019d6fdda7bddd5f65423a4d3`. Le fournisseur est une fixture de test à extraction générique du contexte, non un modèle de langage.

10.3 Modèle de test sans RAG

Prompt: `Please give a short status sentence for this diagnostic request.`. Réponse: `Generated from supplied test context: AUCUNE PREUVE RAG LOCALE N'A ÉTÉ RETROUVÉE.` Statut `UNRESOLVED`; aucun fait externe n'a été inventé.

10.4 Trois questions dans un seul prompt

Prompt: `Which warehouse is assigned to service card XD-620? What is its set point? How often is its inspection interval?`. Réponse: (1) Warehouse Willow Quay; (2) -14.5 C; (3) every 19 days. Hash de sortie composée: `c19d6dce09ea6c8ff958d3d77f278253204ec1785f5b27d8f7b546d840ff8`

10.5 Exemple demandé: capitale du Japon

La fiche de test indépendante contient « the capital city of Japan is Tokyo », hash source: `85a7c7851b` (préfixe). En mode RAG sans modèle, prompt `What is the capital of Japan?`, sortie exacte: `Geography reference JPN-104: the capital city of Japan is Tokyo.`, hash `7f06ba3883681a8c5e40956b` (voir le reçu JSON pour le hash exact s'il diffère de ce préfixe de copie). Avec l'adaptateur de contrat, la sortie préfixée par contexte a le hash `fd337a5e4d3b6dd8ed68099b6404a6bda489e03264dbe0fe2b6079da9940c4a`. Sans RAG, l'adaptateur retourne `UNRESOLVED` avec un message visible d'absence de preuve. Cette expérience vérifie la composition des routes et ne prétend pas que la fixture est un LLM avec connaissances du monde.

11 Phase P8 — restauration et paquetage

Le contrôle final doit partir d'un snapshot, exclure les bytecodes et sorties mutables du manifeste de source, vérifier les hashes/CRC après extraction propre puis lancer le préflight et les tests d'acceptation. Une suite groupée a parfois expiré alors que son test vidéo isolé passe; les deux observations sont conservées, et le timeout groupé reste dans le registre plutôt que d'être effacé.

12 Commandes de reproduction

Depuis la racine de l'archive extraite:

```
python scripts/preflight_o946.py
python scripts/o946_replay_o945_regression.py --project-root . --experiment-dir real_task_tests
python scripts/o946_compound_query_trial.py
python scripts/o946_run_composition_quality_gate.py --project-root . --workspace examples/compo
python scripts/o946_observe_invoice_pipeline.py
python scripts/o946_capture_capital_chat_evidence.py
python scripts/o946_capture_long_generation_evidence.py
```

13 Conclusion et état

Les preuves démontrent des chemins opérationnels bornés: extraction RAG sur des identifiants séparés, orchestration multi-question, conservation de sortie longue via une fixture, compilation et exécution compositionnelles, observation d'une cible externe et rejeu exact sur les entrées vues. Les deux entrées facture hors entraînement restent non résolues; la génération longue et l'exemple Tokyo utilisent des fournisseurs de test, pas un vrai LLM. Le chargement du modèle SmolLM2 de l'utilisateur n'a pas pu être vérifié sans poids et dépendances disponibles. Les exécutions sur le PC de l'utilisateur nécessitent les commandes et instructions livrées; la session ne peut pas observer à distance son ordinateur.